

Algorithmics	Student information	Date	Number of session
	UO: 300737	24/02/2025	2
	Surname: Hoyos	 Escuela de Ingeniería Informática Universidad de Oviedo	
	Name: Ignacio		



Activity 1: Direct Exchange of Bubble Algorithm

TABLE1 = BUBBLE ALGORITHM (times in milliseconds WITHOUT OPTIMIZATION)

n	t ordered	t reverse	t random
10000	317	1 462	1 030
20000	1 242	5 779	4 055
40000	5 137	23 028	16 179
80000	20 607	OoT	OoT
160000	OoT	OoT	OoT

As it was expected, the ordered is quite faster, and the reverse is the slower of the three.

Activity 2: Selection Algorithm

TABLE2 = SELECTION ALGORITHM (times in milliseconds WITHOUT OPTIMIZATION)

n	t ordered	t reverse	t random
10000	307	313	302
20000	1 231	1 258	1 224
40000	4 846	5 072	4 834
80000	19 559	20 070	19 399
160000	OoT	OoT	OoT

I do not expect them to have the same performance, but with a deeper analysis of the selection method, it iterates in all the elements of the vector, searching for the lower value, is not faster or slower if the vector is ordered or not.

Algorithmics	Student information	Date	Number of session
	UO: 300737	24/02/2025	2
	Surname: Hoyos		
	Name: Ignacio		

Activity 3: Insertion Algorithm

TABLE3 = INSERTION ALGORITHM (times in milliseconds WITHOUT OPTIMIZATION)

n	t ordered	t reverse	t random
10000	LoR	298	152
20000	LoR	1 177	591
40000	LoR	4 688	2 345
80000	LoR	18 936	9 405
160000	LoR	OoT	37 380
320000	LoR	OoT	OoT
640000	LoR	OoT	OoT
1280000	LoR	OoT	OoT
2560000	LoR	OoT	OoT
5120000	LoR	OoT	OoT
1024000	LoR	OoT	OoT
2048000	LoR	OoT	OoT
4096000	92	OoT	OoT
8192000	180	OoT	OoT

It makes sense, the ordered will practically do not enter in the while loop as the pivot wouldn't be lower than the previous ones, because of that the random is much slower and the reverse is the worst possible case.

Algorithmics	Student information	Date	Number of session
	UO: 300737	24/02/2025	2
	Surname: Hoyos		
	Name: Ignacio		

Activity 4: Quicksort Algorithm

TABLE4 = QUICKSORT ALGORITHM (times in milliseconds WITHOUT OPTIMIZATION)

n	t ordered	t reverse	t random
250000	LoR	LoR	92
500000	60	73	192
1000000	123	152	401
2000000	259	294	878
4000000	541	625	1883
8000000	1102	1232	4286
16000000	2287	2572	10458

As expected the ordered vector is the fastest, but is a surprise that the random one is worse than the inverted one.

After seeing how long it takes to sort 16 million items initially in random order, calculate and compare (from the complexities and data in the tables above), how many days would each of those three methods (Bubble, Selection and Insertion) take in doing the same?

Algorithm n time RELATION 16 M/n TIME TO SORT 16M TIME IN DAYS

Bubble => 40 000 => 16 719 => 400 => $16\,719 \cdot 400^2 = 2.675.040.000\text{ms}$ => **30,96 days**

Selection => 80 000 => 19 399 => 200 => $19\,399 \cdot 200^2 = 775.960.00\text{ms}$ => **8,98 days**

Insertion => 160 000=> 37 380 => 100 => $37\,380 \cdot 100^2 = 373.800.000\text{ms}$ => **4,33 days**

Algorithmics	Student information	Date	Number of session
	UO: 300737	24/02/2025	2
	Surname: Hoyos		
	Name: Ignacio		

Activity 5: Quicksort + Insertion Algorithm

(There has been a change in the computer use to measure the times from previous exercises)

TABLE4 = QUICKSORT ALGORITHM (times in milliseconds WITHOUT OPTIMIZATION)

Algorithm (n=16000000)	t random
Quicksort	25 160
Quicksort+Insertion (k=5)	24 394
Quicksort+Insertion (k=10)	23 865
Quicksort+Insertion (k=20)	22 799
Quicksort+Insertion (k=30)	22 879
Quicksort+Insertion (k=50)	21 623
Quicksort+Insertion (k=100)	17 938
Quicksort+Insertion (k=200)	10 903
Quicksort+Insertion (k=500)	8 532
Quicksort+Insertion (k=1000)	7 991

We can see a huge improve of the QuickSort algorithm by adding this mixing with Insertion. It seems that the bigger the limit constant, the shorter the time, but obviously it must be a point where it stops to be profitable and the time starts to increase.